Playwright with Pytest

- Install & set up Playwright with Pytest
- Write sync Playwright tests
- Run tests across browsers
- Execute specific tests
- Run tests in parallel

Setup Environment

Prerequisites

- Install **Python**
- Install **Pytest**

Install Playwright with Pytest Plugin

```
pip install pytest-playwright
```

Install Browsers for Playwright

```
playwright install
```

Update Playwright to Latest Version

```
pip install pytest-playwright playwright -U
```

Sync vs Async Support

Playwright provides **both synchronous and asynchronous APIs** in Python.

- Many Playwright operations (like page.goto(), page.title(), page.url()) are **asynchronous** because they interact with the browser.

Keywords in async

- **async** → Declares a function that supports await.
- **await** → Waits for an asynchronous operation to finish before moving to the next step.

👉 **Note**: In Python Playwright, **you don't always need async/await because** it provides **sync API** (playwright.sync_api).

Writing Tests with Playwright and Pytest

```
from playwright.sync_api import Page, expect
import re
def test_verify_page_url(page: Page):
    page.goto("http://www.automationpractice.pl/index.php")
    expect(page).to_have_url("http://www.automationpractice.pl/index.php")

def test_verify_page_title(page: Page):
    page.goto("http://www.automationpractice.pl/index.php")
    expect(page).to_have_title("My Shop")
```

Running Tests (CLI Commands)

Run all tests (default → headless mode):

```
pytest test_playwright.py -s -v
```

Run tests in headed mode:

```
pytest test_playwright.py -s -v --headed
```

Run tests on different browsers:

```
pytest test_playwright.py -s -v --headed --browser firefox
pytest test_playwright.py -s -v --headed --browser chromium
pytest test_playwright.py -s -v --headed --browser chromium --browser firefox
```

Run a specific test:

```
pytest test_playwright.py::test_verify_page_title -s -v
```

Running Tests in Parallel

(Requires **pytest-xdist**)

```
pytest test_playwright.py -s -v --headed --browser chromium -n=2
pytest test_playwright.py -s -v --headed --browser chromium -n 2
pytest test_playwright.py -s -v --headed --browser chromium --numprocesses=2
pytest test_playwright.py -s -v --headed --browser chromium --numprocesses 2
```

List of common HTML elements and their corresponding ARIA roles

HTML Element	ARIA Role
<a href="...">	link
<area href="...">	link
<button>	button
<form> (with name attribute)	form
<h1>, <h2>, <h3>, <h4>, <h5>, <h6>	heading
<header>	banner (context-specific)
<img> (with alt)	img
<input type="button">	button
<input type="checkbox">	checkbox
<input type="email">	textbox
<input type="password">	textbox
<input type="radio">	radio
<input type="range">	slider
<input type="reset">	button
<input type="search">	searchbox
<input type="submit">	button
<input type="text">	textbox
** **	listitem
** , **	list
<option>	option
<select>	listbox (or combobox, depends on usage)
<table>	table
<tbody>, <thead>, <tfoot>	rowgroup
<td>	cell
<th>	columnheader or rowheader
<textarea>	textbox
<tr>	row

Playwright Locators

In Playwright, **locators** are one of the most important concepts.
They provide a **stable and reliable way to find elements on a webpage**.

Why important?

- Locators in Playwright **auto-wait** for elements to be ready.
- They also **retry automatically** if the element is not immediately available.
- This makes your tests less flaky compared to plain CSS or XPath selectors.

Common Built-in Locators

Locator Method	Use Case
page.get_by_role()	Locate elements by accessibility roles like button, checkbox, heading, etc.
page.get_by_text()	Locate by visible text content.
page.get_by_label()	Locate form controls using associated label text.
page.get_by_placeholder()	Locate inputs via placeholder text.
page.get_by_alt_text()	Locate images by their alt attribute.
page.get_by_title()	Locate elements by their title attribute.
page.get_by_test_id()	Locate by custom attribute like data-testid.

Locate by Role

get_by_role() finds elements by how users/assistive technologies see them (e.g., buttons, checkboxes, headings).

Example DOM:

```
<h3>Sign up</h3>

<label><input type="checkbox" /> Subscribe</label><br/>

<button>Submit</button>
```

Test:

```
from playwright.sync_api import Page, expect


def test_locate_by_role(page: Page):
```

```
    page.goto("http://example.com")


    expect(page.get_by_role("heading", name="Sign up")).to_be_visible()
    page.get_by_role("checkbox", name="Subscribe").check()
    page.get_by_role("button", name="Submit").click()
```

✅ Use when: Working with **interactive elements** like buttons, checkboxes, links, tables, etc.

Locate by Label

get_by_label() is best for form fields linked with a <label>.

Example DOM:

```
<label>Password <input type="password" /></label>
```

Test:

```
def test_locate_by_label(page: Page):
    page.goto("http://example.com")
    page.get_by_label("Password").fill("secret123")
```

Locate by Placeholder

get_by_placeholder() finds input fields using placeholder text.

Example DOM:

```
<input type="email" placeholder="name@example.com" />
```

Test:

```
def test_locate_by_placeholder(page: Page):
    page.goto("http://example.com")
    page.get_by_placeholder("name@example.com").fill("playwright@test.com")
```

Locate by Text

get_by_text() searches by visible text content.

Example DOM:

```
<span>Welcome, John</span>
```

Test:

```
def test_locate_by_text(page: Page):
    page.goto("http://example.com")

    # Partial match
    expect(page.get_by_text("Welcome")).to_be_visible()

    # Exact match
    expect(page.get_by_text("Welcome, John", exact=True)).to_be_visible()
```

✅ Use for **non-interactive elements** like , , <p>.
(For clickable elements, prefer role-based locators).

Locate by Alt Text

get_by_alt_text() finds images by their alt attribute.

Example DOM:

```
<img alt="playwright logo" src="/img/playwright-logo.svg" width="100" />
```

Test:

```
def test_locate_by_alt_text(page: Page):
    page.goto("http://example.com")
    page.get_by_alt_text("playwright logo").click()
```

Locate by Title

get_by_title() finds elements using the title attribute.

Example DOM:

```
<span title="Issues count">25 issues</span>
```

Test:

```
def test_locate_by_title(page: Page):
    page.goto("http://example.com")
    expect(page.get_by_title("Issues count")).to_have_text("25 issues")
```

Locate by Test ID

get_by_test_id() works with attributes like data-testid.

Example DOM:

<button data-testid="directions">Directions</button>

Test:

```
def test_locate_by_testid(page: Page):
    page.goto("http://example.com")
    page.get_by_test_id("directions").click()
```

✅ Use when:

- Text or role locators are not reliable.

CSS Locators

What are CSS Selectors?

- **CSS (Cascading Style Sheets)** Selectors in Selenium are used to identify and locate web elements based on their id, class, name, attributes and other attributes.
- **CSS** is a preferred locator strategy as it is simpler to write and faster as compared to XPath.

There are **two main types of selectors**:

- **Absolute CSS Selectors** – Follow the full path to find the element.
- **Relative CSS Selectors** – Find the element using a shortcut path.

Practice on this sample website:
https://testpages.eviltester.com/styled/basic-web-page-test.html

1. Absolute CSS Selectors – Full Path

These selectors go step-by-step from the root (html) to the target element, like following a complete map.

Examples:

What it Selects	Selector
Whole HTML page	html
<head> section	html > head
<title> tag inside head	html > head > title
<body> section	html > body
<h1> inside a div in body	html > body > div > h1
All <p> tags inside nested divs	html > body > div > div > p
<p> with id “para1”	html > body > div > div > p[id='para1']
<p> with class “sub”	html > body > div > div > p[class='sub']
Alternate ways:	
p#para1 = <p> with id=“para1”	html > body > div > div > p#para1
p.sub = <p> with class=“sub”	html > body > div > div > p.sub
<p> with both id and class	html > body > div > div > p[id='para1'][class='main']

Note: Tools like **SelectorsHub** often avoid absolute paths because they’re long and fragile.

2. Relative CSS Selectors – Shortcuts

These selectors find elements directly without tracing the full path.

Examples:

What you want	Selector
<p> inside <body>	body p
<p> anywhere in the HTML	html p
<p> with id="para2"	p[id='para2'] or p#para2
<p> with class="main"	p[class='main'] or p.main
Any element with id="para1"	[id='para1'] or #para1
Any element with class="sub"	[class='sub'] or .sub

Selecting Specific Children

You can select elements based on their **position** among siblings.

Task	Selector
First child inside a div	body > div > *:first-child
Last child inside a div	body > div > *:last-child
3rd child inside a div	body > div > *:nth-child(3)

Attribute Selectors (Pattern Matching)

Use these when you want to match part of an attribute's value.

Task	Selector
Class starts with "ma"	p[class^='ma']
Class ends with "ub"	p[class$='ub']
Class contains "ai"	p[class*='ai']

Combining Selectors

You can combine selectors to filter even more specifically.

Task	Selector
<p> with both id and class	p[id='para1'][class='main']
<p> not having id="para1"	p:not([id='para1'])
<p> with class="sub" but not id="para1"	p:not([id='para1'])[class='sub']
<p> that does NOT have id="para1" AND NOT class="main"	p:not([id='para1']):not([class='main'])

Following Sibling Selectors

Used to select elements that come right after another element.

Task	Selector
<p> right after <p id='para1'>	p[id='para1'] + p
Any tag right after <head>	head + *

Generic way to use CSS in automation:

CSS Selector Type	CSS Format
Tag with ID	tag#id
Tag with Class	tag.classname
Tag with Attribute	tag[attribute="value"]
Tag with Class and Attribute	tag.classname[attribute="value"]

XPath Secrets Every Tester Misses

XML Path Language

1. Absolute XPath

- Full path from the root (html) to the element.

```
/html/body/div[1]/form/input[1]
```

2. Relative XPath

- Starts from anywhere in the DOM (not root).

```
//input[@id="username"]
```

3. Attribute-based XPath

- Uses attributes to locate an element.

```
//button[@name="login"]
```

4. Contains() XPath

- Matches part of an attribute value.

```
//div[contains(@class,"header")]
```

5. Text() XPath

- Locates element using visible text.

```
//button[text()="Submit"]
```

6. Starts-with() XPath

- Matches attribute values that start with specific text.

```
//input[starts-with(@id, "user")]
```

7. Index-based XPath

- When multiple elements match, pick by index.

```
(//input[@type="text"])[1]
```

8. Wildcard (*) XPath

- Selects elements without specifying tag name.

```
//*[@id="loginBtn"]
```

9. OR & AND Conditions

- Combine multiple conditions for precision.

```
//input[@type="text" and
@name="username"]
```

10. XPath with last()

- Selects the last matching element.

```
(//div[@class="item"])[last()]
```

11. XPath with position()

- Selects elements at a specific position.

```
(//tr/td)[position()=3]
```

12. XPath with not()

- Excludes elements with certain conditions.

```
//input[not(@type="hidden")]
```

13. XPath with normalize-space()

- Ignores leading/trailing spaces in text.

```
//button[normalize
space(text())="Login"]
```

14. XPath with ancestor / parent

- Navigate upward in the DOM.

```
//span[@id="child"]/ancestor::div
```

15. XPath with following / preceding

- Navigate sibling relationships.

```
//label[text()="Email"]/following::input[1]
```

16. XPath with following-sibling / preceding-sibling

- Find elements at the same level in DOM.

```
//h2[text()="Section"]/following-sibling::p
```

17. XPath with self

- Selects the current node itself.

```
//div[@class="container"]/self::div
```

18. XPath with descendant

- Selects all children & nested elements.

```
//div[@id="main"]/descendant::a
```

19. XPath with child

- Selects direct child nodes.

```
//ul[@id="menu"]/child::li
```

20. XPath with starts-with() + contains() Combo

- For dynamic attributes that change partially.

```
//input[starts-with(@id, "user")
and contains(@name, "field")]
```

21. Matches with Regex (XPath 2.0)

- For flexible regex-based healing.

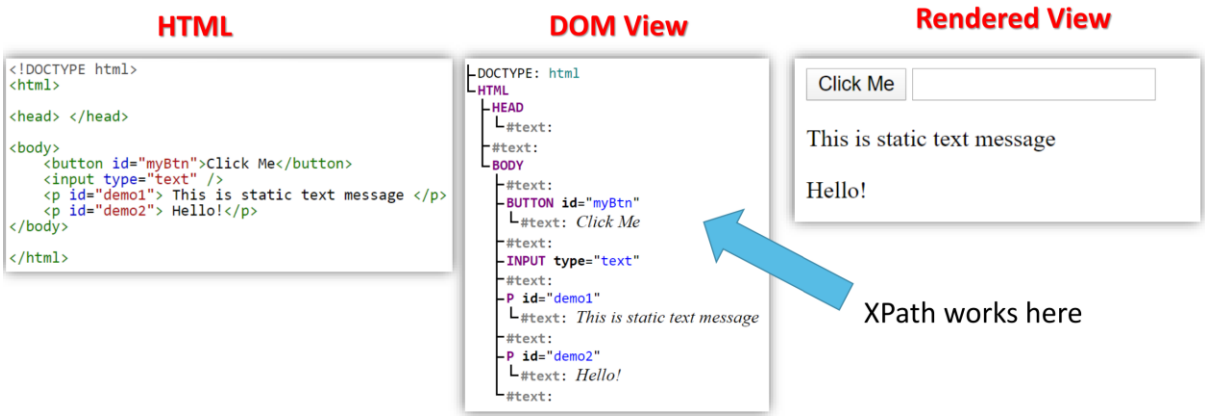
```
//input[matches(@id,'user.*')]
```

XPath

- **XPath** stands for XML Path Language.
- It is a syntax used to navigate through elements and attributes in an XML document.
- In web automation, XPath is used to locate elements on a webpage by their structure and attributes.

DOM (Document Object Model)

- **DOM** represents the structured content of a webpage, with each element as a node in a tree-like structure.
- XPath allows us to locate specific nodes (elements) within the DOM.



Types of XPath

1. Absolute XPath (Full XPath)

- An **Absolute XPath** provides the full path from the root of the document to the target element.
- It starts with a single /, which represents the root node.
- Example: /html/body/header/div/div/div[2]/div/input

2. Relative XPath (Partial XPath)

- A **Relative XPath** is a more flexible way of finding an element. It directly jumps to the element using attributes, without starting from the root.
- It starts with //, which allows XPath to search anywhere in the document.
- Example: //*[@name="search"]

Which XPath Should Be Preferred?

- **Relative XPath** is preferred because it is shorter, easier to maintain, and less likely to break if the webpage structure changes.

Differences Between Absolute and Relative XPaths

Absolute XPath	Relative XPath
Starts with /	Starts with //
Represents the full path from the root node	Does not need the full path, can directly target elements
Does not use attributes	Primarily works with attributes like id, class, etc.
Traverses through every node in the DOM	Jumps directly to the element using attributes

XPath Usage:

https://demowebshop.tricentis.com/

1. Absolute XPath

- Refers to the complete path from the root element to the target element in the DOM structure.
- **XPath Format**: /html/body/div/div[2]/div[1]
- **Example XPath**: /html[1]/body[1]/div[4]/div[1]/div[1]/div[1]/a[1]/img[1]
- **WebElement Example**: Logo

Notes: Absolute XPath is very specific and less reliable because even a small change in the webpage structure can break the XPath.

2. Relative XPath

- A shorter, more flexible version of XPath that begins from any node in the DOM, rather than the root.
- **XPath Format**: //tagname[@attribute='value']
- **Example XPath**: //img[@alt='Tricentis Demo Web Shop']
- **WebElement Example**: Logo

Notes: Relative XPath is more commonly used as it adapts better to minor changes in the page structure.

3. XPath with Single Attribute

- Selects elements based on one attribute and its value.
- **XPath Format**: //tagname[@attribute='value']
- **Example XPath**: //input[@id='small-searchterms']
- **WebElement Example**: Search box

4. XPath with Multiple Attributes

- Selects elements by matching multiple attributes and their respective values.
- **XPath Format**: //tagname[@attribute1='value1'][@attribute2='value2']
- **Example XPath**: //input[@value='Search'][@type='submit']
- **WebElement Example**: Search button

Notes: Multiple attributes help make the XPath more specific and reliable.

5. XPath with *and* Operator

- Uses the ***and*** operator to combine multiple attribute conditions that must all be true for the element to be selected.
- **XPath Format**: //tagname[@attribute1='value1' and @attribute2='value2']
- **Example XPath**: //input[@value='Search' and @type='submit']
- **WebElement Example**: Search button

6. XPath with *or* Operator

- Uses the ***or*** operator to combine multiple attribute conditions, where only one of the conditions needs to be true for the element to be selected.
- **XPath Format**: //tagname[@attribute1='value1' or @attribute2='value2']
- **Example XPath**: //input[@value='Search' or @type='submit']
- **WebElement Example**: Search button

7. XPath with contains() Function

- Matches elements that contain a specific substring within an attribute.
- **XPath Format**: //*[contains(@class, 'btn')]
- **Example XPath**: //h2//a[contains(@href,'computer')]

- **WebElement Example**: Selects products whose name contains "computer" under Featured Products.

Note: The contains() function is useful for partial matches.

8. XPath with starts-with() Function

- Matches elements whose attribute values start with a specified string.
- **XPath Format**: //*[starts-with(@id, 'user')]
- **Example XPath**: //h2//a[starts-with(@href,'/build')]
- **WebElement Example**: Selects products whose name starts with "build" under Featured Products.

Note: The starts-with() function is helpful for dynamic elements whose IDs or classes are partially consistent.

9. XPath with text() Function

- Selects elements based on the exact text content of the element.
- **XPath Format**: //*[text()='Login']
- **Example XPath 1**: //*[text()="Register"] – Selects the Register link.
- **Example XPath 2**: //*[contains(text(), "Fiction")] – Selects all product titles under Books that contain "Fiction".

10. XPath with last() Function

- Selects the last element in a set of matching nodes.
- **XPath Format**: //input[last()]
- **Example XPath**: //div[@class='column follow-us']//li[last()]
- **WebElement Example**: Selects Google+ under the FOLLOW US footer menu.

Note: The last() function is useful when you want the last occurrence of an element.

11. XPath with position() Function

- Selects an element based on its position in the list of matching nodes.
- **XPath Format**: //input[position()=2]
- **Example XPath**: //div[@class='column follow-us']//li[position()=2]

- **WebElement Example**: Selects the 2nd option under the FOLLOW US footer menu.

Dynamic XPath

```
<button name="start" class="start" style="">START</button>
```

```
<button name="stop" class="stop" style="">STOP</button>
```

XPath	Technique
//button[@name='start']	Exception
//button[@name='stop']	Exception
//button[@name='start' or @name='stop']	or
//button[contains(@name,'st')]	contains()
//button[starts-with(@name,'st')]	starts-with()
//button[text()='STOP' or text()='START'];	text() - innertext

Lab Exercise - Dropdown

Lab Exercise 1: Sorting Products and Printing Prices using Playwright with Pytest

1. **Open the Website**
    - Navigate to bstackdemo.com
2. **Locate and Verify Dropdown**
    - Locate the **"Order by"** dropdown element.
    - Assert that it is **visible** and **enabled.**
3. **Sort Products**
    - Select the **"Lowest to highest"** option from the dropdown.
    - Wait for the page to reflect the sorting changes.
4. **Extract Product Data**
    - Capture all product prices and product names using locators:
        - prices
        - product names
5. **Print Products and Prices**
    - Loop through all products and print names with corresponding prices.
6. **Identify Lowest & Highest Priced Products**
    - Print the **first product** as the lowest priced
    - Print the **last product** as the highest priced

Expected Output

- **All product names and their prices** printed in ascending order of price.
- **Lowest-priced product** printed first with its price.
- **Highest-priced product** printed last with its price.

inner_text(), text_content(), all_inner_texts(), all_text_contents(), and all()

1️⃣ inner_text() vs text_content()

Both methods retrieve text from a **single element** – but there are key differences:

Method	What It Does	Key Points	When to Use
inner_text()	Returns the **visible text** of the element	✅ Ignores hidden elements ✅ Removes extra whitespaces ✅ Removes line breaks	When you want **clean, user-visible text** for validation
text_content()	Returns the **raw text content** of the element.	❌ Includes hidden text ❌ Keeps extra spaces, tabs, and line breaks 🔄 Returns None if element has no text	When you want **exact text** including hidden/whitespace.

Example:

```
product = page.locator(".product-title").nth(1)

print(product.inner_text())     # Clean visible text: '14.1-inch Laptop'

print(product.text_content())   # Raw text with spaces: '\n   14.1-inch Laptop  \n'
```

2️⃣ all_inner_texts() vs all_text_contents()

Both methods return **text from multiple elements at once**, as a **list of strings**.

Method	What It Does	Key Points
all_inner_texts()	Returns **visible text** of each matched element (cleaned)	✅ Removes whitespace/line breaks ✅ Ignores hidden text
all_text_contents()	Returns **raw text content** of each matched element	❌ Keeps whitespaces and line breaks ❌ Includes hidden text

Example:

```
products = page.locator(".product-title")
print(products.all_inner_texts())
```

Output: ['14.1-inch Laptop', 'Build your own cheap computer', 'Digital Storm ...']

```
print(products.all_text_contents())
```

Output: ['\n 14.1-inch Laptop \n', '\n Build your own cheap computer \n', ...]

You can clean the result manually:

```
product_names = [text.strip() for text in products.all_text_contents()]
print(product_names)
```

3 all() – Converting Locator to List of Locators

The all() method **converts a Locator into a list of individual locators**.
This allows you to iterate over them and perform actions one by one.

Method	What It Does
locator.all()	Returns a **list of Locator objects** (one per element)

Example:

```
products = page.locator(".product-title")
product_locators = products.all()  # Now we have a Python list of Locators
print(product_locators)  # [<Locator ...>, <Locator ...>, <Locator ...>]
```

```
# Loop through each locator
for product in product_locators:
   print(product.inner_text())
```

This is useful when:

- You need to **click each element** one by one.
- You want to **perform different actions** per element (not just read all text at once).
- You want to **combine with index-based logic** (e.g., click the 2nd, 4th, and 5th element only).

Summary Table

Concept	Returns	Cleans Whitespace?	Includes Hidden Text?	Best Use Case
inner_text()	Single string	✅ Yes	❌ No	Get visible text of one element
text_content()	Single string (or None)	❌ No	✅ Yes	Get raw/unprocessed text
all_inner_texts()	List of strings	✅ Yes	❌ No	Get visible text of multiple elements
all_text_contents()	List of strings	❌ No	✅ Yes	Get raw text of multiple elements
all()	List of Locators	N/A	N/A	Iterate and act on elements individually

Lab Assignments

Lab 1: Handling Dynamic Web Tables

URL: https://testautomationpractice.blogspot.com/

Objective: Extract and compare process data from a dynamic web table.

Test Scenarios:

- Retrieve the **CPU Load** value for the **Chrome** process and compare it against the value displayed in the **yellow** label.
- Retrieve the **Memory Usage** value for the **Firefox** process and compare it against the value displayed in the **blue** label.
- Retrieve the **Network Speed** value for the **Chrome** process and compare it against the value displayed in the **orange** label.
- Retrieve the **Disk Space** value for the **Firefox** process and compare it against the value displayed in the **violet** label.

Lab 2: Extract Data from a Paginated Table

URL: https://testautomationpractice.blogspot.com/

Objective: Read and print data from a table that uses pagination.

ID	Name	Price	Select
1	Smartphone	$10.99	☐
2	Laptop	$19.99	☐
3	Tablet	$5.99	☐
4	Smartwatch	$7.99	☐
5	Wireless Earbuds	$8.99	☐

1 2 3 4

Lab 3: BlazeDemo – Flight Booking Automation

Step 1: Launch the Website

- Navigate to https://blazedemo.com/

Step 2: Select Departure and Destination

- Select "Boston" as the **departure** city and "London" as the **destination** using the dropdown options.

Step 3: Search for Flights

- Click on the **Find Flights** button after selecting cities.

Step 4: Capture Flight Prices

- Locate the results table and extract flight prices.
- Store all prices in an array and print the total number of available flights.

Step 5: Identify the Lowest Price

- Sort the array of prices and determine the flight with the lowest fare.

Step 6: Choose the Cheapest Flight

- Find the corresponding table row with the lowest price and click **Choose This Flight**.

Step 7: Enter Passenger Information

- Fill in the booking form with the following details:
    - **Name:** John
    - **Address:** 1403 American Beauty Ln
    - **City:** Columbus
    - **State:** OH
    - **Zip Code:** 43240
    - **Credit Card Number:** 6789 0673 4523 1267
    - **Credit Card Year:** 2023
    - **Name on Card:** John Canedy
- Click the **Purchase Flight** button.

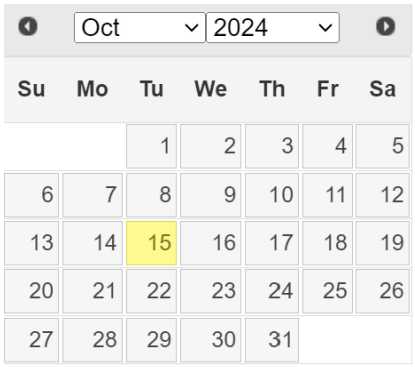
Step 8: Confirm Purchase

- Validate the success message: *"Thank you for your purchase"*.
- Print **"Success !! Passed"** if the message appears; otherwise, print **"Failed"**.

Lab Assignments

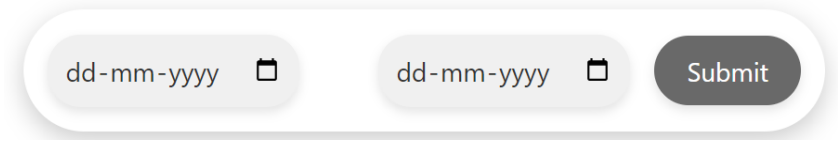
Lab Assignment 1: jQuery Date Picker (Contains Drop Downs)

https://testautomationpractice.blogspot.com/



Lab Assignment 2: Select Date Range (using fill())

https://testautomationpractice.blogspot.com/



Lab Assignment 3: Dummy Ticket Booking

Action Steps:

1. Setup:

- Goto to URL: https://www.dummyticket.com/dummy-ticket-for-visa-application/

2. Select Ticket Type:

- Select "Dummy ticket for Visa Application" radio button.

3. Enter Passenger Details:

- Enter first name: "Akash".
- Enter last name: "Ratore".
- Open the date picker for Date of Birth and select: Year: "2001", Month: "Mar", Date: "2".
- Select sex: "Male".

4. Enter Travel Details:

- Select trip type: "One-way".
- Enter departure city: "Toronto".
- Enter destination city: "Mumbai".
- Open the date picker for departure date and select: Year: "2024", Month: "Nov", Date: "25".

5. Additional Information:

- Enter notes: "Need visa as soon as possible".

6. Delivery Options:

- Select purpose of ticket: "Visa Application".
- Open the date picker for appointment date and select: Year: "2024", Month: "Dec", Date: "10".
- Select delivery method: "Email".

7. Enter Billing Details:

- Enter name: "Akash Rathore".
- Enter phone: "+12345678956".
- Enter email: "abc.123@gmail.com".
- Select country: "Canada".
- Enter address: "123 Scott Street, Niagara Falls, Ontario, L2C 6M1".

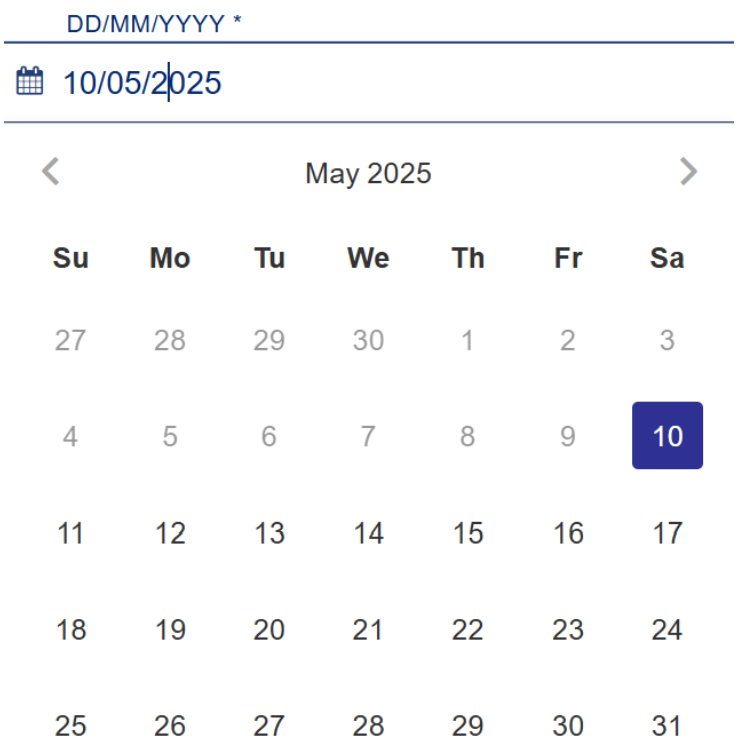
8. Verify Product details table

Product	Quantity	Total
× Dummy ticket for Visa Application	1	₹1,200
Cart Subtotal		**₹1,200**
Order Total		**₹1,200**

9. Place order (Do not proceed with the payment).

Lab Assignment: 4: IRCTC

URL: https://www.irctc.co.in/nget/train-search

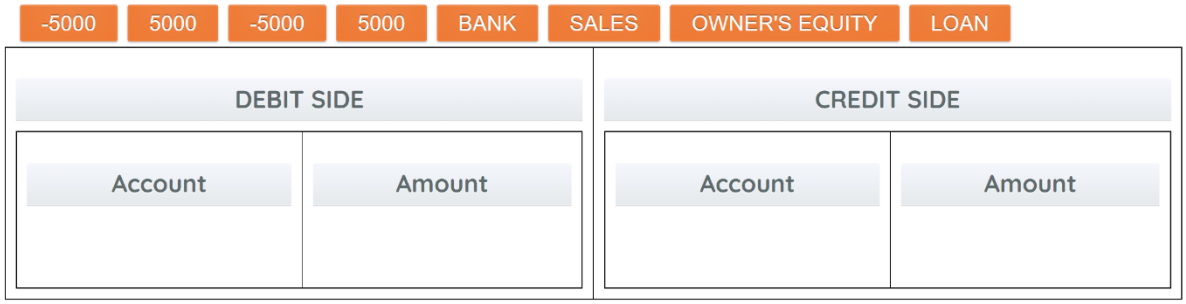


Lab Assignments

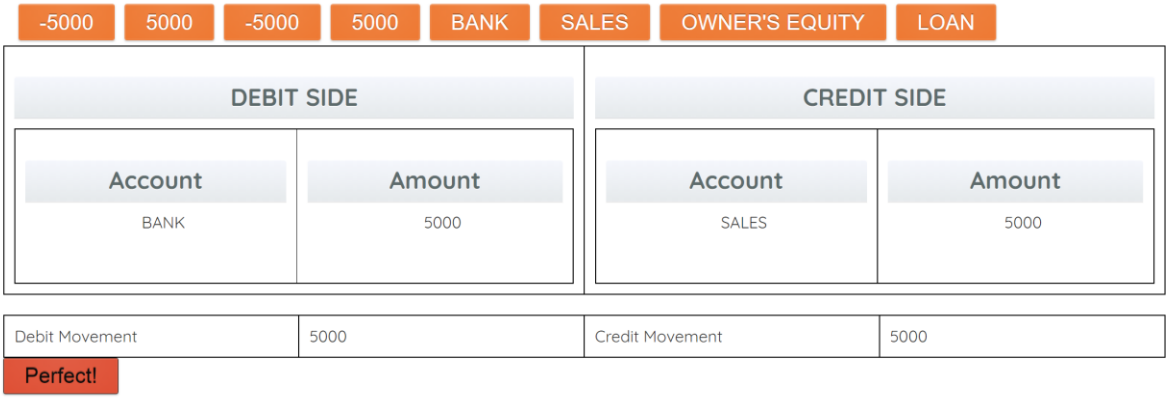
Assignment 1: Drang and Drop

http://demo.guru99.com/test/drag_drop.html

Before Drag and Drop



After Drag and Drop



Playwright- Screenshots, Videos, and Tracing (Trace Viewer)

🖼️ Screenshots in Playwright (with Pytest)

Screenshots help capture the state of your web page during test execution.

1. Capture a Screenshot

To take a screenshot of the current browser view:

```
page.screenshot(path='screenshot.png')
```

2. Capture a Full Page Screenshot

Takes a screenshot of the **entire scrollable area** of the web page:

```
page.screenshot(path='fullpage.png', full_page=True)
```

3. Capture a Specific Element

You can also capture a screenshot of a **particular element**:

```
page.locator('.header').screenshot(path='header.png')
```

⚙️ Configure Screenshots in pytest.ini

You can automatically control screenshot behavior from pytest.ini:

```
[pytest]
addopts = --screenshot=on
```

Option	Description
on	Always capture screenshots after tests
off	Disable screenshots
only-on-failure	Capture screenshots only when a test fails
on-first-failure	Capture only on the first failed test

🎬 Video Recording in Playwright (with Pytest)

Playwright can **record test execution videos** — useful for debugging visual issues or understanding test behavior.

Through Code

Example:

```
def test_video_recording(playwright):
    browser = playwright.chromium.launch()
    context = browser.new_context(record_video_dir="videos/")  # Folder to save videos
    page = context.new_page()
    page.goto("https://example.com")
    page.click("text=More information")
    context.close()  # Video is saved when context is closed
    browser.close()
```

⚙️ Configure Videos in pytest.ini

You can manage video recording behavior in pytest.ini:

[pytest]
addopts = --video=on

Option	Description
off	Disable video recording
on	Record all tests
retain-on-failure	Keep videos only for failed tests
on-first-retry	Record video only when a test is retried

🔍 Tracing and Trace Viewer

Playwright **Tracing** is a debugging tool that records everything happening during a test — including:

- User actions
- Network requests
- DOM snapshots
- Console logs

You can **replay and inspect** these traces later using **Trace Viewer**.

1. Start and Stop Tracing in Code

```
def test_tracing_example(playwright):
    browser = playwright.chromium.launch(headless=False)
    context = browser.new_context()
    # Start tracing with screenshots and snapshots
    context.tracing.start(screenshots=True, snapshots=True)
    page = context.new_page()
    page.goto("https://demowebshop.tricentis.com/")
    page.click("text=Login")
    # Stop tracing and save it as a zip file
    context.tracing.stop(path="trace.zip")
    browser.close()
```

⚙️ **Enable Tracing in pytest.ini**

You can enable tracing globally from **pytest.ini**:

```
[pytest]
addopts = --tracing=on
```

Option	Description
on	Record trace for each test
off	Disable tracing (default)
retain-on-failure	Keep traces only for failed tests

Enable Tracing from Command Line

You can also control tracing through CLI:

```
pytest --tracing=on
```

🧿 **Viewing the Trace File (trace.zip)**

Once tracing is enabled, Playwright generates a .zip file that contains the test trace.
You can view this trace in **three ways**:

✅ **Option 1 — Command Line**

```
playwright show-trace trace.zip
```

🌐 **Option 2 — Online Viewer**

Visit: https://trace.playwright.dev/
Then drag and drop your trace.zip file.

Flaky Tests and Retries in Pytest with Playwright

What Are Flaky Tests?

Flaky tests **sometimes pass and sometimes fail** even when the code hasn't changed — often due to:

- Slow network or animations
- Dynamic page elements
- Timing issues

Such tests make results unreliable.

✅ Playwright Retries (Pytest Style)

You can retry failed tests automatically in **pytest** using the **pytest-rerunfailures** plugin.

1. Install Plugin

```
pip install pytest-rerunfailures
```

2. Use CLI command

```
pytest prep/test_flaky.py --reruns 3 --reruns-delay 2
```

→ This retries failed tests **up to 3 times** with a **2-second delay** between each retry.

Example Scenario

Attempt	Result	Outcome
1	❌ Failed	Retry triggered
2	✅ Passed	Marked as flaky

Data-Driven Testing (DDT) in Pytest with Playwright

What is Data-Driven Testing?

Data-Driven Testing (DDT) is a testing approach where a single test case runs multiple times with different input data.
Instead of writing separate tests for each input, you **parameterize** the test with data sets.

In **Pytest**, this is achieved using the **@pytest.mark.parametrize** decorator.
In **Playwright**, the same logic is combined with browser automation to test UI workflows like login, search, or form submission.

Why Use Data-Driven Testing?

- Improves test coverage
- Reduces code duplication
- Makes tests maintainable
- Allows testers to separate **data** from **test logic**

Different Data Sources for DDT

Below are the common ways to provide test data to Pytest-Playwright tests.

1️⃣ Using Hardcoded Data (List or Tuple)

This is the simplest and fastest way to start data-driven testing.

Example:

```
import pytest
from playwright.sync_api import expect

# Test data defined in a list
test_data = [
    ("user1@example.com", "pass123", "valid"),
    ("wrong@example.com", "wrongpass", "invalid"),
]

@pytest.mark.parametrize("email, password, validity", test_data)
def test_login(page, email, password, validity):
    page.goto("https://demowebshop.tricentis.com/login")
    page.fill("#Email", email)
    page.fill("#Password", password)
    page.click('input[value="Log in"]')
    # Add assertions based on validity
```

📌 **Use Case:** Quick demo or when data is small and fixed.
📌 **Pros:** Very simple — no external files.
📌 **Cons:** Hard to scale; test data lives inside code.

2️⃣ Using JSON File

Data can be stored in a .json file and read dynamically.

Example:

data.json

```
[
 {"email": "valid@example.com", "password": "test123", "validity": "valid"},
 {"email": "wrong@example.com", "password": "abc123", "validity": "invalid"}
]
```

Test:

```
import json, pytest

with open("testdata/data.json") as f:
    login_data = json.load(f)

@pytest.mark.parametrize("email, password, validity", [
    (data["email"], data["password"], data["validity"]) for data in login_data
])
def test_login(page, email, password, validity):
    # Steps and assertions here
```

📌 **Use Case:** Structured and reusable test data.
📌 **Pros:** Easy to extend, machine-readable.
📌 **Cons:** Not suitable for tabular or large datasets.

3️⃣ Using CSV File

CSV files are great for tabular test data that manual testers can easily edit.

Example:

data.csv

```
email,password,validity
user1@example.com,pass123,valid
wrong@example.com,wrongpass,invalid
```

Test:

```
import csv, pytest

data = []
with open("testdata/data.csv") as file:
    reader = csv.DictReader(file)
    for row in reader:
        data.append((row["email"], row["password"], row["validity"]))

@pytest.mark.parametrize("email, password, validity", data)
def test_login(page, email, password, validity):
    # Steps and assertions here
```

📌 **Use Case:** Large tabular data.
📌 **Pros:** Editable via Excel or Google Sheets.
📌 **Cons:** All values are text; no nested data.

4️⃣ Using Excel File (openpyxl Library)

Excel is popular among testers for managing test cases and data.

Concept:

The **openpyxl** library reads Excel files row by row.
You can skip header rows and handle blank cells safely.

Example:

Excel file (data.xlsx):

email	password	validity
user1@example.com	pass123	valid
wrong@example.com	wrongpass	invalid

Test:

```
from openpyxl import load_workbook
import pytest

data = []
wb = load_workbook("testdata/data.xlsx")
sheet = wb.active

for row in sheet.iter_rows(min_row=2, values_only=True):
```

```
    email, password, validity = row
    data.append((str(email or ""), str(password or ""), str(validity or "")))

@pytest.mark.parametrize("email, password, validity", data)
def test_login(page, email, password, validity):
    # Steps and assertions here
```

📌 **Use Case:** When test data is managed in Excel sheets.
📌 **Pros:** Familiar to testers, organized layout.
📌 **Cons:** Requires openpyxl installation and handling of blank cells.

Common Test Flow in All Examples

1. **Load data** from a chosen source (List, JSON, CSV, Excel).
2. **Pass data** to test using @pytest.mark.parametrize.
3. **Perform actions** with Playwright (fill(), click(), etc.).
4. **Validate results** dynamically (e.g., success message, error message).

Summary Comparison

Source	Library	File Format	Ideal For	Advantages
List/Tuple	None	N/A	Small static data	Simple, fast setup
JSON	json	.json	Structured data	Flexible, readable
CSV	csv	.csv	Tabular data	Easy to maintain
Excel	openpyxl	.xlsx	Spreadsheet data	User-friendly for testers

Reporting

Pytest HTML Test Report in Playwright (using pytest-html)

Overview

pytest-html is a plugin that helps generate **beautiful HTML reports** for your Playwright tests. These reports show which tests passed, failed, or were skipped — and you can even attach screenshots for failed tests.

Official documentation: https://pypi.org/project/pytest-html/

Step 1: Install the plugin

Run the following command in your terminal or command prompt:

```
pip install pytest-html
```

Step 2: Configure pytest.ini

Add these lines to your **pytest.ini** file (in the project root folder):

```
[pytest]
addopts = --headed
    --browser=chromium
    --video=retain-on-failure
    --screenshot=only-on-failure
    --tracing=retain-on-failure
    --html=myreport.html --self-contained-html --capture=tee-sys
```

Explanation:

- --headed → Runs the browser in UI mode.
- --browser=chromium → Specifies the browser to use.
- --video, --screenshot, --tracing → Captures test evidence on failure.
- --html=myreport.html → Generates an HTML report file.
- --self-contained-html → Embeds CSS and JS into one file (no external dependencies).
- --capture=tee-sys → Displays logs in both console and report.

Step 3: Attach Screenshots on Test Failures

Add the following code to your **conftest.py** file.
It captures a screenshot whenever a test fails and attaches it to the HTML report.

```
from pathlib import Path

import pytest

from slugify import slugify


@pytest.hookimpl(hookwrapper=True)

def pytest_runtest_makereport(item, call):

    pytest_html = item.config.pluginmanager.getplugin("html")

    outcome = yield

    screen_file = ''

    report = outcome.get_result()

    extra = getattr(report, "extra", [])


    if report.when == "call":

        xfail = hasattr(report, "wasxfail")

        if report.failed or xfail and "page" in item.funcargs:

            page = item.funcargs["page"]

            screenshot_dir = Path("screenshots")

            screenshot_dir.mkdir(exist_ok=True)

            screen_file = str(screenshot_dir / f"{slugify(item.nodeid)}.png")

            page.screenshot(path=screen_file)


        if (report.skipped and xfail) or (report.failed and not xfail):

            extra.append(pytest_html.extras.png(screen_file))

        report.extra = extra
```

Tip:

- The screenshot will be saved inside a folder named **screenshots**.
- The image will also appear in the HTML report for easy viewing.

Step 4: Create Sample Tests

Example test file: **test_sample.py**

```
from playwright.sync_api import Page, expect

def test_google_search(page):
    page.goto("https://www.google.com")
    expect(page).to_have_title("Google")


def test_bing_search(page):
    page.goto("https://www.bing.com/")
    expect(page).to_have_title("Bing123")  # Intentionally failed
```

Step 5: Run the Tests and Generate Report

Run your tests using:

```
pytest
```

This will generate a file named **myreport.html** in your project folder.

Optional: Add a Timestamp to the Report Name

You can add the current date and time to your report name when running tests via command line.

For mac/Linux:

```
pytest test_sample.py --html=reports/report_$(date +"%Y-%m-%d_%H-%M-%S").html --self-contained-html
```

For Windows (PowerShell):

```
pytest test_sample.py --html="report_$(Get-Date -Format 'yyyy-MM-dd_HH-mm-ss').html" --self-contained-html
```

Allure Test Report (using pytest) in Playwright

Overview

Allure Report is a powerful and visually appealing reporting tool for test automation frameworks like **Pytest**.
It shows detailed information such as test steps, logs, screenshots, and execution history in an interactive dashboard.

Step 1: Install Allure Pytest Plugin

Run the following command to install the required plugin:

```
pip install allure-pytest
```

Step 2: Install Allure Command-Line Tool

The **Allure CLI** (Command-Line Interface) is needed to view and generate reports.

For Windows:

1. Go to https://github.com/allure-framework/allure2/releases
2. Download the latest .zip file.
3. Extract it and **add the bin folder** path to your **system PATH environment variable**.

For macOS:

```
brew install allure
```

For Linux:

```
sudo snap install allure --classic
```

Step 3: Configure Allure in pytest.ini

Add the following line to your pytest.ini file:

```
[pytest]

addopts = --alluredir=reports/allure-results
```

Explanation:

- **--alluredir** specifies the folder where test execution results will be stored (in JSON format).
- Later, these results will be converted into an HTML report.

Step 4: Attach Screenshots on Test Failures (conftest.py)

```
from pathlib import Path
import pytest
from slugify import slugify
import allure

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    pytest_html = item.config.pluginmanager.getplugin("html")
    outcome = yield
    screen_file = ''
    report = outcome.get_result()
    extra = getattr(report, "extra", [])
    if report.when == "call":
        xfail = hasattr(report, "wasxfail")
        if report.failed or xfail and "page" in item.funcargs:
            page = item.funcargs["page"]
            screenshot_dir = Path("screenshots")
            screenshot_dir.mkdir(exist_ok=True)
            screen_file = str(screenshot_dir / f"{slugify(item.nodeid)}.png")
            page.screenshot(path=screen_file)

        if (report.skipped and xfail) or (report.failed and not xfail):
            # add the screenshots to the html report
            extra.append(pytest_html.extras.png(screen_file))

            # Attach to Allure report
            allure.attach.file(
                screen_file,
                name="Failure Screenshot",
                attachment_type=allure.attachment_type.PNG
            )
        report.extra = extra
```

Step 5: Create and Run Your Tests

Write a few Playwright tests as usual.

Example:

```
from playwright.sync_api import Page, expect

def test_google_search(page):

    page.goto("https://www.google.com")

    expect(page).to_have_title("Google")
```

```
def test_bing_search(page):
    page.goto("https://www.bing.com")
    expect(page).to_have_title("Bing123")  # Intentionally failed
```

Run your tests using:

```
pytest
```

After the test run, Allure will create a folder:

```
reports/allure-results
```

Step 6: Generate and View the Allure Report

There are two ways to view the report:

Option 1: Temporary report (auto-opens in browser)

```
allure serve reports/allure-results
```

This will **generate and launch** the report in your browser automatically.
(Report will not be saved permanently.)

Option 2: Permanent report (saved for future reference)

```
allure generate reports/allure-results -o reports/allure-report --clean
```

Then open it manually:

```
allure open reports/allure-report
```

Combine Reports (e.g., HTML + Allure)

You can combine multiple report formats in one run:

```
pytest --html=/report.html --junitxml=reports/results.xml --alluredir=reports/allure-results
```

HTML Report

Step 1: Install the plugin

Step 2: Configure pytest.ini

Step 3: Attach Screenshots on Test Failures (conftest.py)

Step 4: Create Sample Tests

Step 5: Run the Tests and Generate Report

Allure report

Step 1: Install Allure Pytest Plugin

Step 2: Install Allure Command-Line Tool (to view and generate reports)

Step 3: Configure Allure in pytest.ini

Step 4: Attach Screenshots on Test Failures (conftest.py)

Step 5: Create and Run Your Tests

Step 6: Generate and View the Allure Report

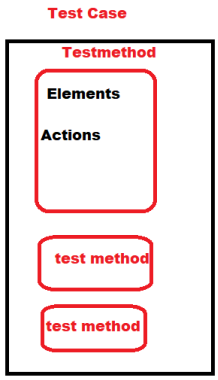
Page Object Model (POM) in Playwright

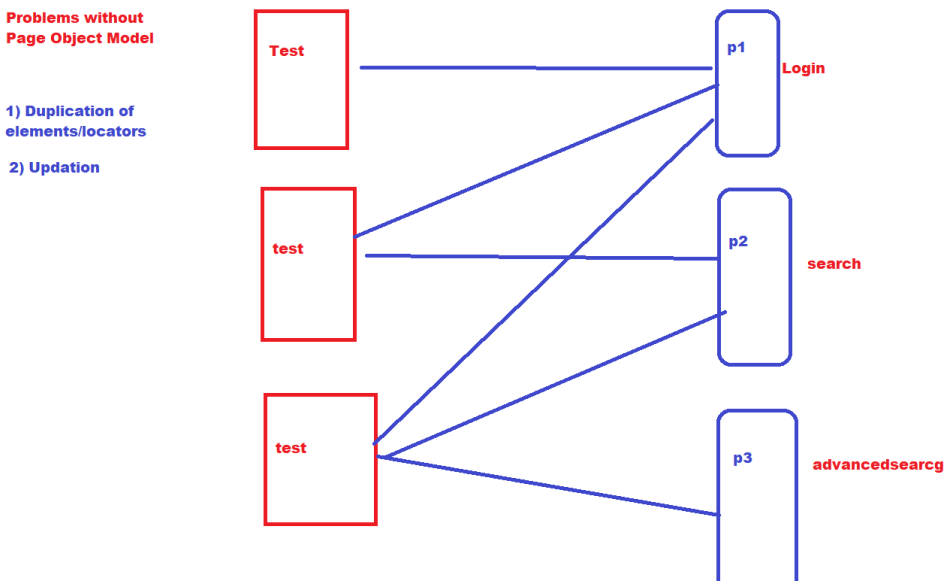
Page Object Model (POM) is a **design pattern** in test automation that helps in creating **maintainable, reusable, and readable** test code by separating the test logic from the **UI interactions**.

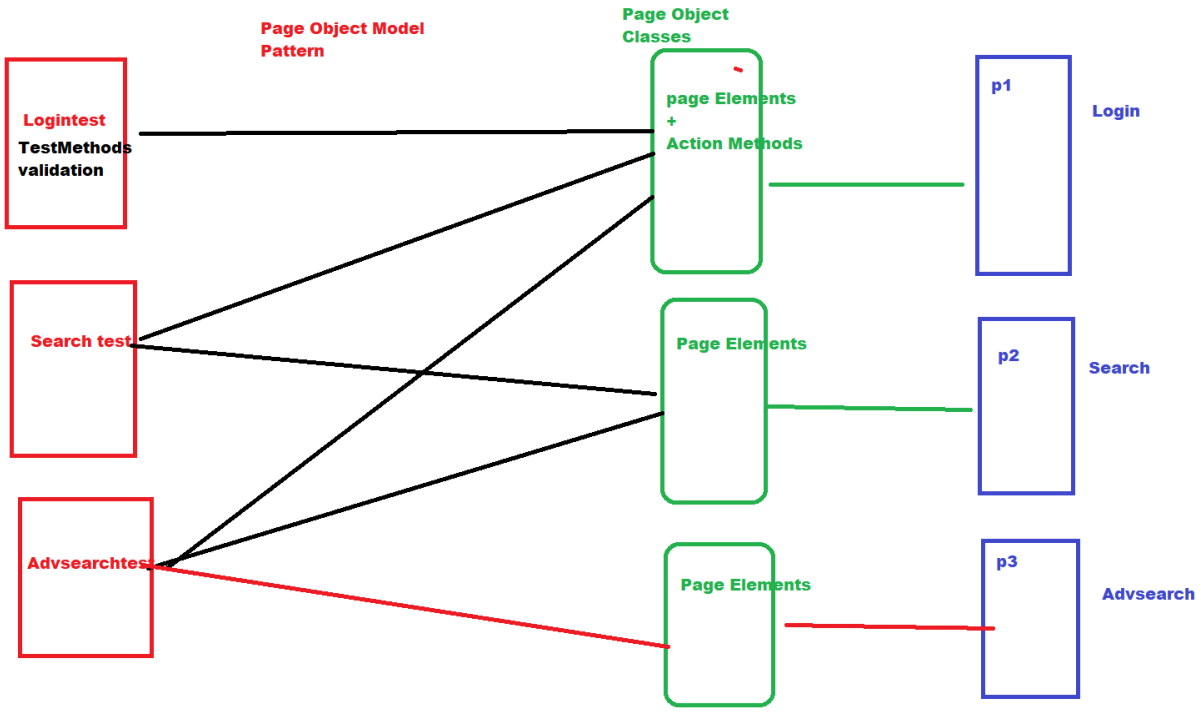
- Each web page is represented as a **class**.
- The **elements** on the page are represented as **variables**.
- The **actions** on the elements (like click, fill, etc.) are written as **methods**.

Why Use POM in Playwright?

- **Code reusability**: Page functions can be reused across test files.
- **Easy maintenance**: If UI changes, only the corresponding page file needs an update.
- **Better readability**: Clear separation of test logic and page structure.
- **Scalability**: Ideal for large-scale projects with many test cases.







Example:

Page Object Class (search.py)

```
class SearchPage:
    def __init__(self, page):
        self.page = page
        self.search_term_input = page.locator('[aria-label="Enter your search term"]')

    def navigate(self):
        self.page.goto("https://bing.com")

    def search(self, text):
        self.search_term_input.fill(text)
        self.search_term_input.press("Enter")
```

test_search.py

```
from search import SearchPage

# in the test
page = browser.new_page()
search_page = SearchPage(page)
search_page.navigate()
search_page.search("search query")
```